WARWICK
THE UNIVERSITY OF WARWICK

Lecture 4
Advanced Language Models

Gabriele Pergola

Natural Language Processing
CS918

January 2026

Outline

- Generalisation of Language Models
- Smoothing Approaches for Generalisation
  - Laplace smoothing
  - Interpolation and backoff
  - Good Turing Smoothing
  - Kneser-Ney Smoothing

Generalisation of Language Models

Estimating Bigram Probabilities

- **Maximum Likelihood Estimate (MLE):**

$$P(w_i \mid w_{i-1}) = \frac{count(w_{i-1}, w_i)}{count(w_{i-1})}$$

e.g.

$$P(the \mid in) = \frac{count(\text{"in the"})}{count(\text{"in"})}$$

Generalisation of Language Models

- **Language models:**

  P(I, am, fine) → high
  P(am, fine, I) → low or 0

- **Remember:** language models will work best on similarly looking data.

  If we train on social media data, that may not work for novels.

  - OMG I'm ROTFL! → may be frequent in social media, unlikely in novels!

Generalisation of Language Models

- **Limitation:** We're assuming that all n-grams in new, unseen data will have been observed in the training data.
  - Is this the reality though?

- We need to **consider the possibility of new n-grams** in unseen data.

Shakespeare's Bigrams

- Shakespeare used:
  - N = 884,647 tokens
  - V = 29,066 types → $V^2 \approx 845M$ possible bigrams!

- Shakespeare only produced ~300,000 bigram types (**4% of all possible bigrams!**)
  - Other bigrams may be possible, but we haven't observed them.

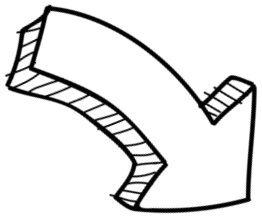
Zeros

Training set

- … found a penny
- … found a solution
- … found a tenner
- … found a book

Test set

- … found a fiver
- … found a card



P(card | found a) = 0

The Intuition of Smoothing

- We have sparse statistics:

  P(w | “found a”)

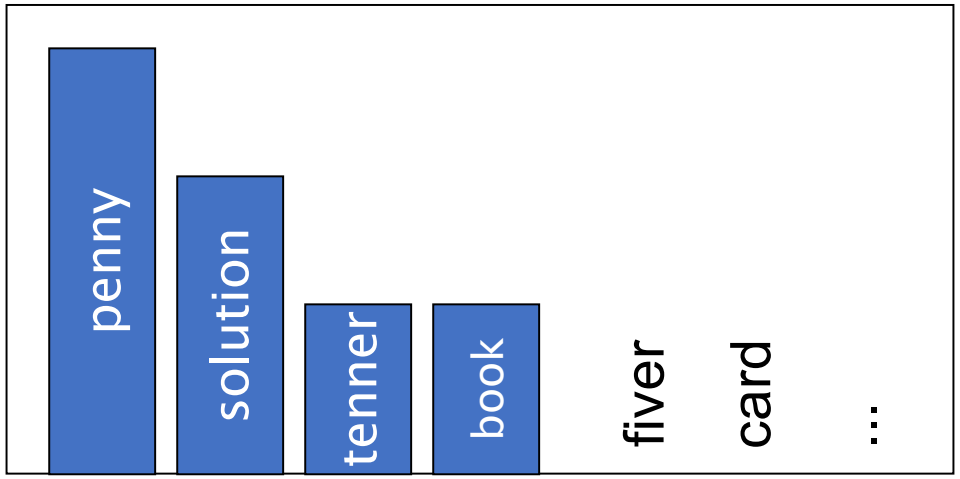
  3 → penny

  2 → solution

  1 → tenner

  1 → book

  7 -- total



The Intuition of Smoothing

- We'd like to improve the distribution:

  P(w | "found a")

  3 → *penny* → 2.5

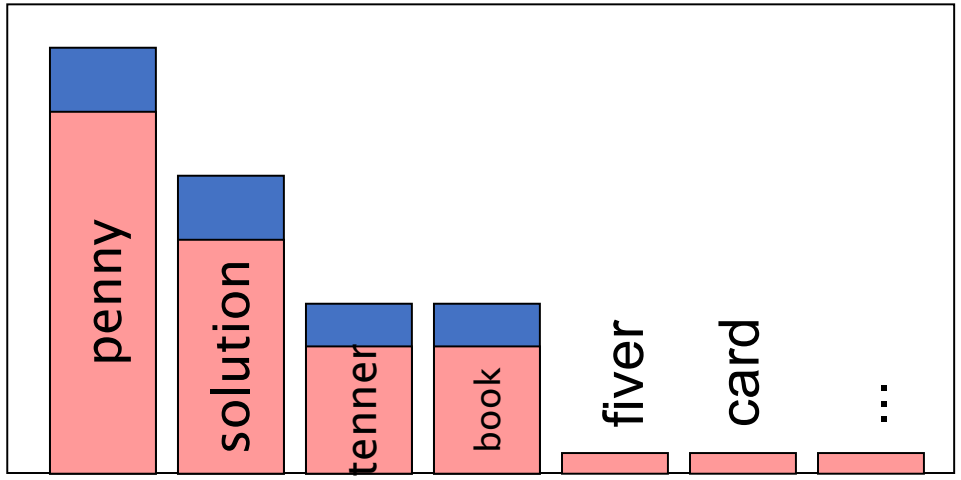
  2 → *solution* → 1.5

  1 → *tenner* → 0.5

  1 → *book* → 0.5

  other → 2

  7 → total



Four Possible Solutions

- We'll see 4 possible solutions:
  1) Laplace smoothing
  2) Interpolation and backoff
  3) Good Turing Smoothing
  4) Kneser-Ney Smoothing

Laplace Smoothing

Laplace Smoothing: Add-One Estimation

- Pretend we have seen **each word one more time** than we actually did.

- MLE estimate: $$P_{MLE}(\,found \mid you\,) = \frac{count(\,you\ found\,)}{count(\,you\,)}$$

Laplace Smoothing: Add-One Estimation

- Pretend we have seen **each word one more time** than we actually did.

- MLE estimate:

$$P_{MLE}(w_i \mid w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

- **Add-one estimate**:

$$P_{Add-1}(w_i \mid w_{i-1}) = \frac{c(w_{i-1}, w_i) + 1}{c(w_{i-1}) + V}$$

Sample of Bigram Counts

- Remember our sample of **bigrams** counted in a corpus of **9,222 sentences**

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

"i" is followed by another "i" on 5 occasions

"to" is followed by "eat" on 686 occasions

Laplace Smoothed Bigram Counts

Add 1 to ALL values in the matrix

	i	want	to	eat	chinese	food	lunch	spend
i	6	828	1	10	1	1	1	3
want	3	1	609	2	7	7	6	2
to	3	1	5	687	3	1	7	212
eat	1	1	3	1	17	3	43	1
chinese	2	1	1	1	1	83	2	1
food	16	1	16	1	2	5	1	1
lunch	3	1	1	1	1	2	1	1
spend	2	1	2	1	1	1	1	1

Laplace Smoothed Bigram Probabilities

$$P^*(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n)+1}{C(w_{n-1})+V}$$

	i	want	to	eat	chinese	food	lunch	spend
i	0.0015	0.21	0.00025	0.0025	0.00025	0.00025	0.00025	0.00075
want	0.0013	0.00042	0.26	0.00084	0.0029	0.0029	0.0025	0.00084
to	0.00078	0.00026	0.0013	0.18	0.00078	0.00026	0.0018	0.055
eat	0.00046	0.00046	0.0014	0.00046	0.0078	0.0014	0.02	0.00046
chinese	0.0012	0.00062	0.00062	0.00062	0.00062	0.052	0.0012	0.00062
food	0.0063	0.00039	0.0063	0.00039	0.00079	0.002	0.00039	0.00039
lunch	0.0017	0.00056	0.00056	0.00056	0.00056	0.0011	0.00056	0.00056
spend	0.0012	0.00058	0.0012	0.00058	0.00058	0.00058	0.00058	0.00058

Reconstituted Counts

- Instead of changing both the numerator and denominator, it is convenient to describe how a smoothing algorithm affects the numerator, by defining an **adjusted count C^*.**

$$P^*(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n)+1}{C(w_{n-1})+V} = \frac{C(w_{n-1}w_n)^*}{C(w_{n-1})}$$

$$C(w_{n-1}w_n)^* = \frac{[C(w_{n-1}w_n)+1]\times C(w_{n-1})}{C(w_{n-1})+V}$$

Reconstituted Counts

$$c^*(w_{n-1}w_n) = \frac{[C(w_{n-1}w_n)+1] \times C(w_{n-1})}{C(w_{n-1})+V}$$

	i	want	to	eat	chinese	food	lunch	spend
i	3.8	527	0.64	6.4	0.64	0.64	0.64	1.9
want	1.2	0.39	238	0.78	2.7	2.7	2.3	0.78
to	1.9	0.63	3.1	430	1.9	0.63	4.4	133
eat	0.34	0.34	1	0.34	5.8	1	15	0.34
chinese	0.2	0.098	0.098	0.098	0.098	8.2	0.2	0.098
food	6.9	0.43	6.9	0.43	0.86	2.2	0.43	0.43
lunch	0.57	0.19	0.19	0.19	0.19	0.38	0.19	0.19
spend	0.32	0.16	0.32	0.16	0.16	0.16	0.16	0.16

Comparing Original vs. Smoothed

- Original:

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

dividing by 10!

- Smoothed:

	i	want	to	eat	chinese	food	lunch	spend
i	3.8	527	0.64	6.4	0.64	0.64	0.64	1.9
want	1.2	0.39	238	0.78	2.7	2.7	2.3	0.78
to	1.9	0.63	3.1	430	1.9	0.63	4.4	133
eat	0.34	0.34	1	0.34	5.8	1	15	0.34
chinese	0.2	0.098	0.098	0.098	0.098	8.2	0.2	0.098
food	6.9	0.43	6.9	0.43	0.86	2.2	0.43	0.43
lunch	0.57	0.19	0.19	0.19	0.19	0.38	0.19	0.19
spend	0.32	0.16	0.32	0.16	0.16	0.16	0.16	0.16

TOO MUCH PROBABILITY MASS WAS MOVED TO ALL THE ZEROS!

More General Laplace Smoothing: Add-k

$$P_{Add-k}(w_i \mid w_{i-1}) = \frac{c(w_{i-1}, w_i) + k}{c(w_{i-1}) + kV}$$

Laplace Smoothing is not Ideal for LMs

- It's generally **not the best solution for language models**.
  - With such a sparse matrix, we're **replacing too many 0's with 1's**.

- It's still **often used for NLP tasks**, particularly when we observe fewer 0's.

Interpolation and Backoff

Backoff and Interpolation

- Different words may need different context size captured

  e.g. I may identify that:

  ”**United States**” needs to be captured as a **bigram**…

  ...but…

  ”**look forward to**” is better captured as a **trigram**

Backoff and Interpolation

- **Backoff:**
  - use *trigram* if it's very common,
  - otherwise *bigram*, otherwise *unigram*

  Difficult to implement, define "very common"

- **Interpolation:**
  - mix all unigram, bigram, trigram

Linear Interpolation

- Simple interpolation:

$$\hat{P}(w_n|w_{n-2}w_{n-1}) = \lambda_1 P(w_n) + \lambda_2 P(w_n|w_{n-1}) + \lambda_3 P(w_n|w_{n-2}w_{n-1})$$

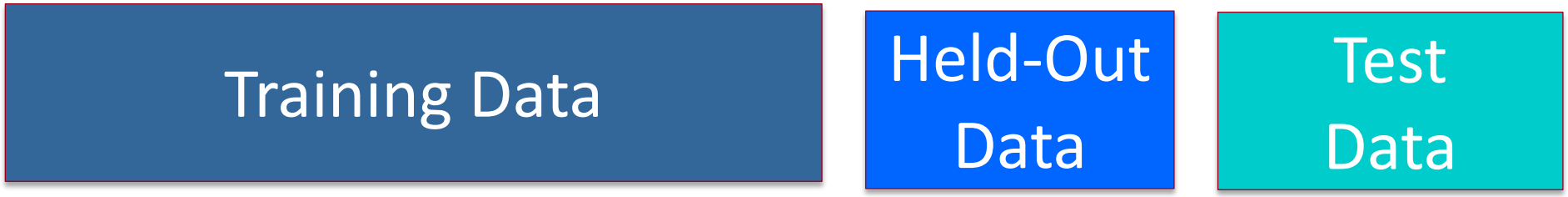
$$\sum_i \lambda_i = 1$$

- Lambdas **conditional on context**:

$$\hat{P}(w_n|w_{n-2}w_{n-1}) = \lambda_1(w_{n-2:n-1})P(w_n) + \lambda_2(w_{n-2:n-1})P(w_n|w_{n-1}) + \lambda_3(w_{n-2:n-1})P(w_n|w_{n-2}w_{n-1})$$

How do we set those Lambdas?

- Use a held-out (or development) corpus:



- Fix the N-gram probabilities (on the **training data**)
- Then, search for λs that give largest probability (or lowest perplexity) on the **held-out set**:

$$\log P(w_1 \ldots w_n \mid M(\lambda_1 \ldots \lambda_k)) = \sum_i \log P_{M(\lambda_1 \ldots \lambda_k)}(w_i \mid w_{i-1})$$

Open vs. Closed Vocabulary Tasks

- Two scenarios when processing held out or test data:

  1) We know ALL words in advance.

  - i.e. all words were also in training data.
  - We have a fixed vocabulary V – **closed vocab. Task**

  2) We **may find new, unseen words** (generally the case)

  - Out Of Vocabulary (OOV) words – **open vocab. task**

How do we deal with OOV words?

- We can use a token for all unknown words: **<OOV>**
- How to train:
  - Create a **fixed lexicon L** of size V.
  - While preprocessing text, change **words not in L to <OOV>**
  - Train the LM, where <OOV> is just another token.

- In test: new words are assigned the probability of <OOV>.

Recap - Four Possible Solutions

- So far, 2 possible solutions:
  1) Laplace smoothing
  2) Backoff and Interpolation
  3) Good Turing Smoothing
  4) Kneser-Ney Smoothing

Recap - Four Possible Solutions

- So far, 2 possible solutions:
  1) **Laplace smoothing**
  2) Backoff and Interpolation
  3) Good Turing Smoothing
  4) Kneser-Ney Smoothing

$$P(\text{“penny”} \mid \text{“found a”}) = \frac{P(\text{“found a penny”})}{P(\text{“found a”})}$$

Recap - Four Possible Solutions

- So far, 2 possible solutions:
  1) **Laplace smoothing**
  2) Backoff and Interpolation
  3) Good Turing Smoothing
  4) Kneser-Ney Smoothing

$$P(\text{“penny”} \mid \text{“found a”}) = \frac{\overset{0}{P(\text{“found a penny”})}}{P(\text{“found a”})}$$

Recap - Four Possible Solutions

- So far, 2 possible solutions:
  1) **Laplace smoothing**
  2) Backoff and Interpolation
  3) Good Turing Smoothing
  4) Kneser-Ney Smoothing

$$P(\text{“penny”} \mid \text{“found a”}) = \frac{P(\text{“found a penny”}) + 1}{P(\text{“found a”}) + |V|}$$

Recap - Four Possible Solutions

- So far, 2 possible solutions:
  1) Laplace smoothing
  2) **Backoff and Interpolation**
  3) Good Turing Smoothing
  4) Kneser-Ney Smoothing

$$P(\text{“penny”} \mid \text{“found a”}) = \frac{P(\text{“found a penny”})}{P(\text{“found a”})}$$

0

Recap - Four Possible Solutions

- So far, 2 possible solutions:
  1) Laplace smoothing
  2) **Backoff and Interpolation**
  3) Good Turing Smoothing
  4) Kneser-Ney Smoothing

$$P(\text{“penny”} \mid \text{“found a”}) = \frac{P(\text{“found a penny”})}{P(\text{“found a”})}$$

From trigrams to bigrams

$$P(\text{“penny”} \mid \text{“found a”}) = \frac{P(\text{“a penny”})}{P(\text{“a”})}$$

Recap - Four Possible Solutions

- So far, 2 possible solutions:
  1) Laplace smoothing
  2) **Backoff and Interpolation**
  3) Good Turing Smoothing
  4) Kneser-Ney Smoothing

$$P(\text{“penny”} \mid \text{“found a”}) = \frac{P(\text{“found a penny”})}{P(\text{“found a”})}$$

From trigrams to bigrams

$$P(\text{“penny”} \mid \text{“found a”}) = \frac{P(\text{“a penny”})}{P(\text{“a”})}$$

$$P(\text{“penny”} \mid \text{“found a”}) \approx P(\text{“penny”} \mid \text{“a”})$$

Recap - Four Possible Solutions

- So far, 2 possible solutions:
  1) Laplace smoothing
  2) **Backoff and Interpolation**
  3) Good Turing Smoothing
  4) Kneser-Ney Smoothing

Better: linear interpolation

A better solution is to mix all probabilities: unigram, bigram, trigram, etc.

For this, we need scalar positive weights $\lambda_0, \lambda_1, \ldots, \lambda_{n-1}$ such that $\sum_i \lambda_i = 1$

$$\hat{P}(\text{“penny”} \mid \text{“found a”}) \approx \lambda_2 P(\text{“penny”} \mid \text{“found a”})$$
$$\lambda_1 P(\text{“penny”} \mid \text{“a”})$$
$$\lambda_0 P(\text{“a”})$$

Good Turing Smoothing

Intuition of Good Turing Smoothing

- If your dataset has many words seen **once**, what does that say about **unseen** words?

- There are probably many unseen words!

- Intuition of Good Turing Smoothing:

The **frequency** of rare words tells us how likely unseen words are.

Intuition of Good Turing Smoothing

- To estimate counts of n-grams we've **never seen**

  → use the count of n-grams **we've seen once**

⇓

Frequency
of
Frequency

Frequency of Frequency "*C*"

- N_c = count of tokens observed *C* times
- e.g. I am happy and I am fine and I have to go

I	→ 3
am	→ 2
and	→ 2
happy	→ 1
fine	→ 1
have	→ 1
to	→ 1
go	→ 1

$N_1 = 5$

$N_2 = 2$

$N_3 = 1$

Frequency of Frequency

Frequency of Frequency "*C*"

- N_c = count of tokens observed *C* times
- e.g. I am happy and I am fine and I have to go

I	→ 3
am	→ 2
and	→ 2
happy	→ 1
fine	→ 1
have	→ 1
to	→ 1
go	→ 1

$N_1 = 5$

$N_2 = 2$

$N_3 = 1$

Frequency of Frequency

Frequency of Frequency "*C*"

- N_c = count of tokens observed *C* times
- e.g. I am happy and I am fine and I have to go

I	→ 3
am	→ 2
and	→ 2
happy	→ 1
fine	→ 1
have	→ 1
to	→ 1
go	→ 1

$N_1 = 5$

$N_2 = 2$

$N_3 = 1$

Frequency of Frequency

Intuition of Good Turing Smoothing

- You have seen the following **12** occurrence of words:

I	→ 3	happy	→ 1
am	→ 2	fine	→ 1
and	→ 2	have	→ 1
		to	→ 1
		go	→ 1

- How likely is it that the next word is **to**?

  1/12 → easy, we have the probability directly

- How likely is it that the next word is new, e.g. **bye**?
  - Use our **count of things-we-saw-once**
    5 elements seen once (*happy, fine, have, to, go*) → 5/12
  - → OK, but **probabilities won't add up to 1** now!
    we need to revise probabilities.

Normalization - Good Turing Smoothing

- Let space $\boldsymbol{S_r}$ be the total number of occurrences of n-grams that appear $\boldsymbol{r}$ times in the training data

I	→ 3
am	→ 2
and	→ 2
happy	→ 1
fine	→ 1
have	→ 1
to	→ 1
go	→ 1

$N_1 = 5$ → $\boldsymbol{S_1 = 5} = r\,N_r = 1 * 5$

$N_2 = 2$

$N_3 = 1$

Normalization - Good Turing Smoothing

- Let $\boldsymbol{S_r}$ be the total number of occurrences of n-grams that appear $\boldsymbol{r}$ times in the training data

I	→ 3
am	→ 2
and	→ 2
happy	→ 1
fine	→ 1
have	→ 1
to	→ 1
go	→ 1

$N_1 = 5$

$N_2 = 2$

$N_3 = 1$

$\boldsymbol{S_1 = 5} = r\,N_r = 1 * 5$

$\boldsymbol{S_2 = 4} = r\,N_r = 2 * 2$

Normalization - Good Turing Smoothing

- Let $\boldsymbol{S_r}$ be the total number of occurrences of n-grams that appear $\boldsymbol{r}$ times in the training data

I	→ 3
am	→ 2
and	→ 2
happy	→ 1
fine	→ 1
have	→ 1
to	→ 1
go	→ 1

$N_1 = 5$

$N_2 = 2$

$N_3 = 1$

$\boldsymbol{S_1 = 5} = r\,N_r = 1 * 5$

$\boldsymbol{S_2 = 4} = r\,N_r = 2 * 2$

$\boldsymbol{S_3 = 3} = r\,N_r = 3 * 1$

Normalization - Good Turing Smoothing

S_r

- S_r is calculated as the product of $r * N_r$

  r → the number of times an n-gram appears

  N_r → the number of different n-grams that appear exactly *r* times

- The reason for this is that if you have N_r different n-grams each occurring *r* times, the **total occurrences of all these n-grams** is rN_r

- This total is referred to as the "**space**" S_r because it represents the collective share of all occurrences in the corpus for n-grams that appear *r* times.

Normalization - Good Turing Smoothing

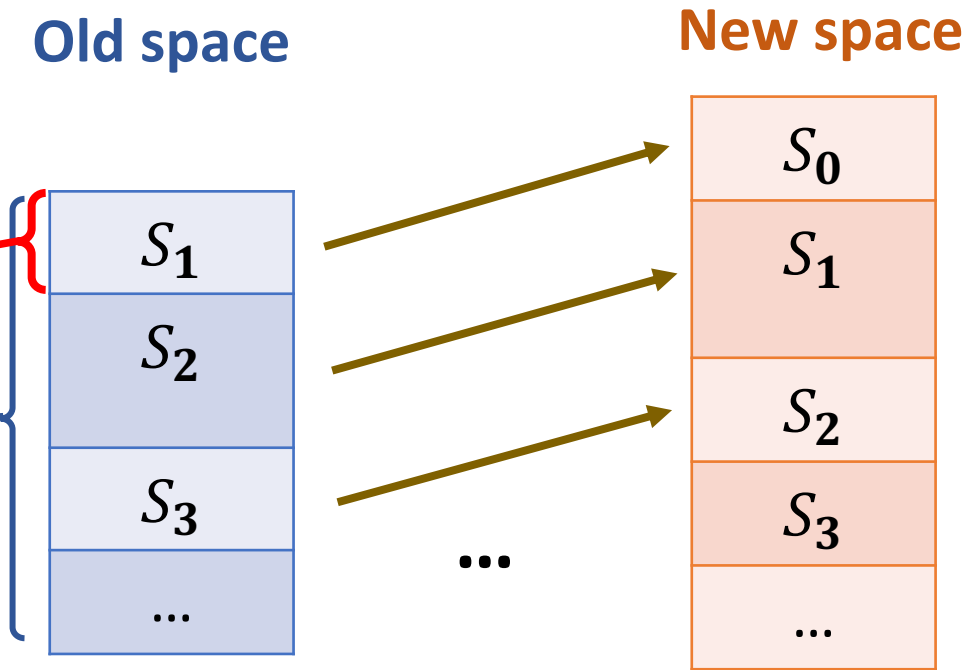
- Let $\boldsymbol{S_r}$ be the total number of n-grams that occur $\boldsymbol{r}$ times in the training data

Proportion of probability space occupied by n-grams in $\boldsymbol{S_r}$ in the **new** space

$=$

Proportion of probability space occupied by n-grams in $\boldsymbol{S_{r+1}}$ in the **old** space

- Spread evenly among all n-grams in S_r



The **proportion** (i.e. probability) with regards to the **whole** (N) is:

$$\frac{S_r}{N} = \frac{(r)N_r}{N}$$

Formula for Good-Turing

- N_r be the number of different n-grams observed exactly r times
  - E.g.: "I am happy and I am fine and I have to go", then $N_1 = 5$ and $N_2 = 2$
  - In notation on previous slide, $\boldsymbol{rN_r}$ is the size of $\boldsymbol{S_r}$
- **Good-Turing:** Probability for any n-gram with r occurrences (in the new space) is estimated from the old space occupied by n-grams with occurrences $r + 1$
- Let N be the size of the training data, the probability space (i.e. *proportion*) occupied by n-grams with count $r + 1$ is: $\frac{(r+1)N_{r+1}}{N}$
- Share this mass evenly among n-grams observed r times, there are N_r of them:

$$\frac{(r+1)N_{r+1}}{N} \times \frac{1}{N_r}$$

- This is the Good–Turing probability assigned to **one specific n-gram** whose observed count is r:

$$P_{GT}(c = r) = \frac{(r+1)N_{r+1}}{N \times N_r}$$

Good Turing Calculations

$$P_{GT}(c = r) = \frac{(r+1)N_{r+1}}{N} \times \frac{1}{N_r}$$

I	→ 3	happy	→ 1
am	→ 2	fine	→ 1
and	→ 2	have	→ 1
		to	→ 1
		go	→ 1

$N_1 = 5$
$N_2 = 2$
$N_3 = 1$

Old space

r	N_r
$S_1 = 1 \times N_1 = 5$	
$S_2 = 2 \times N_2 = 4$	
$S_3 = 3 \times N_3 = 3$	
…	

…

New space

S_0 $(N_0 = 1)$
S_1
S_2
S_3
…

$$P_{GT}(c = 0) = \frac{(0+1)N_{0+1}}{N \times N_0} = \frac{N_1}{N} = \frac{5}{12}$$

$$P_{GT}(c = 1) = \frac{(1+1)N_{1+1}}{N \times N_1} = \frac{2 \times N_2}{N \times N_1}$$

$$= \frac{2 \times 2}{12 \times 5} = \frac{0.8}{12}$$

N_r be the number of different n-grams occurred exactly r times

Good Turing Calculations

$$P_{GT}(c = r) = \frac{(r+1)N_{r+1}}{N} \times \frac{1}{N_r}$$

I	→ 3	happy	→ 1
am	→ 2	fine	→ 1
and	→ 2	have	→ 1
		to	→ 1
		go	→ 1

$N_1 = 5$
$N_2 = 2$
$N_3 = 1$

Old space

r	N_r
$S_1 = 1 \times N_1 = 5$	
$S_2 = 2 \times N_2 = 4$	
$S_3 = 3 \times N_3 = 3$	
…	

…

New space

S_0 $(N_0 = 1)$
S_1
S_2
S_3
…

$$P_{GT}(c = 0) = \frac{(0+1)N_{0+1}}{N \times N_0} = \frac{N_1}{N} = \frac{5}{12}$$

$$P_{GT}(c = 1) = \frac{(1+1)N_{1+1}}{N \times N_1} = \frac{2 \times N_2}{N \times N_1}$$

$$= \frac{2 \times 2}{12 \times 5} = \frac{0.8}{12}$$

N_r be the number of different n-grams occurred exactly r times

Intuition of Good Turing Smoothing

- You have seen the following **12** occurrence of words:

I	→ 3	happy	→ 1
am	→ 2	fine	→ 1
and	→ 2	have	→ 1
		to	→ 1
		go	→ 1

- How likely is it that the next word is **to**?

  1/12 → easy, we have the probability directly

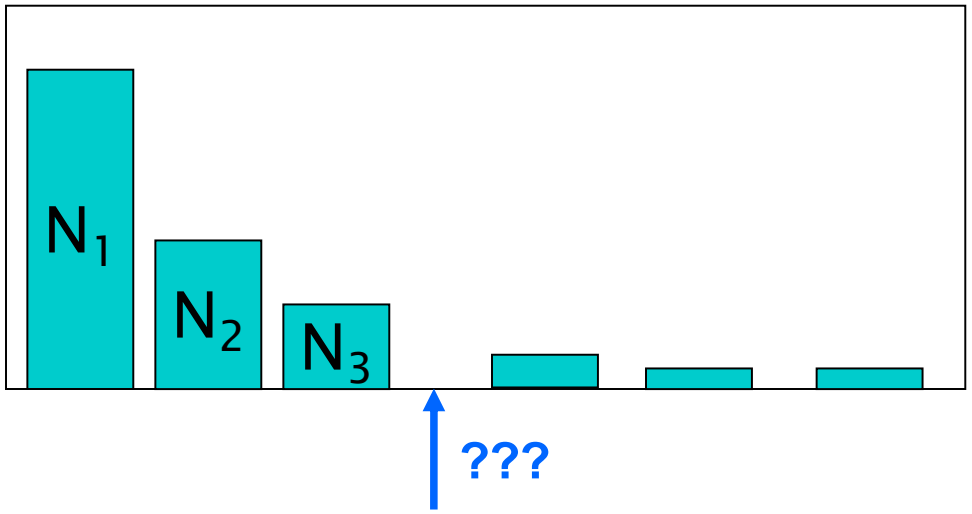
- How likely is it that the next word is new, e.g. **bye**?
  - Use our **count of things-we-saw-once**

    5 elements seen once (*happy, fine, have, to, go*) → **5/12**
  - → OK, but **probabilities won't add up to 1** now!
    we need to revise probabilities.

Good Turing Issues

- **Problem:** what about the word "*and*"? (say its count $r = 12156$)
  - For small count r, $N_r > N_{r+1}$
  - For large count r? $N_{r+1} = N_{12157}$ will likely be zero.

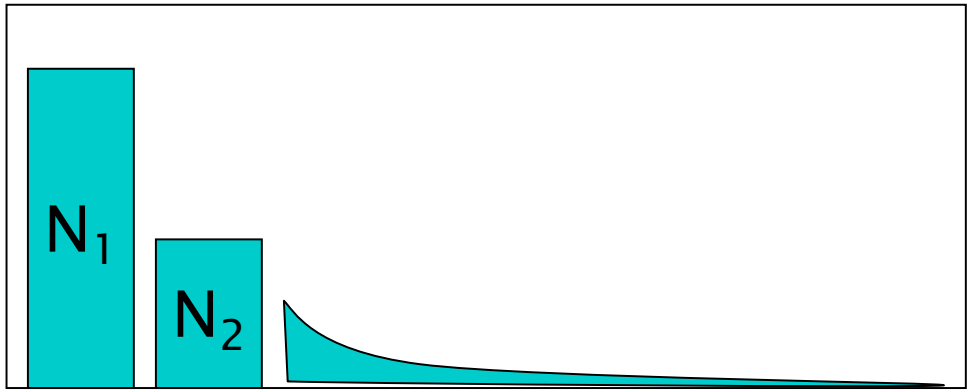
$N_1 = 5$
$N_2 = 2$
$N_3 = 1$
$N_4 = ?$
$N_5 = 1$



Good Turing Issues

- Solution to avoid zeros for large values of count:
  - Simple Good-Turing [Gale and Sampson]:
    replace N_r → best-fit power law for unreliable counts

$$F(r) = ar^b$$



Search for the best a and $b < -1$ to fit observed N_r

Good Turing Numbers → Kneser-Ney

Corpus with 22 million words of AP Newswire

- For each original count ***c***, the Good-Turing method provides a smoothed count ***c∗***.

- *E.g.: if an n-gram was originally observed 0 times, the Good-Turing estimate **c∗** pretends we saw it 0.0000270 times.*

$$P_{GT}(c = r) = \frac{c^*}{N}$$

$$c^* = \frac{(r+1)N_{r+1}}{N_r}$$

Count c	Good Turing c*
0	.0000270
1	0.446
2	1.26
3	2.24
4	3.24
5	4.22
6	5.19
7	6.21
8	7.24
9	8.25

Good Turing Numbers → Kneser-Ney

Corpus with 22 million words of AP Newswire

- If you look at it, for $c \geq 2$:
  - $\boldsymbol{c^* \approx c - 0.75}$

  Can we avoid the hassle of doing all Turing calculations?

Count c	Good Turing c*
0	.0000270
1	0.446
2	1.26
3	2.24
4	3.24
5	4.22
6	5.19
7	6.21
8	7.24
9	8.25

Kneser-Ney Smoothing

Absolute Discounting Interpolation

- Save ourselves some time and **just subtract 0.75** (or some **d**)!

Interpolation weight

discounted bigram

$$P_{\text{AbsoluteDiscounting}}(w_i \mid w_{i-1}) = \frac{c(w_{i-1}, w_i) - d}{c(w_{i-1})} + \lambda(w_{i-1})P(w)$$

unigram

Kneser-Ney Smoothing

- We need to predict the next word:
  - I can't see without my reading _____?
  - BUT: we have never seen "reading X" in training data.
    - Backoff, rely only on unigrams?
  - If we only look at unigrams, P("Kingdom") > P("glasses").
    - Is it "reading Kingdom" then?
  - **Intuition of Kneser-Ney smoothing:**
    "Kingdom" ALWAYS comes after "United"
    why would it ever come after "reading"?

Kneser-Ney Smoothing

- In other words, $P_{continuation}$ (w) is:
  - The number of **word types** seen to precede *w* normalised by the number of distinct words preceding all words

I am
We are
You are
They are
She is

$P_{continuation}$("is") = 1/5

Kneser-Ney Smoothing

- In other words, $P_{continuation}$ (w) is:
  - The number of **word types** seen to precede *w* normalised by the number of distinct words preceding all words

I am
We are
You are
They are
She is

$P_{continuation}$("are") = 3/5

Kneser-Ney Smoothing

- In other words, $P_{continuation}$ (w) is:
  - The number of **word types** seen to precede *w* normalised by the number of distinct words preceding all words

I am
We are
You are
They are
She is

$P_{continuation}$("am") = 1/5

Kneser-Ney Smoothing

- Instead of P(w): "How likely is w"

  $P_{continuation}(w)$: "*How likely is w to appear as a novel continuation?*"

- For each unigram, count the number of bigram types it completes → i.e. the number of different contexts word *w* has appeared in

Kneser-Ney Smoothing

- How many times does w appear as a novel continuation:

$$P_{CONTINUATION}(w) \propto \left|\{w_{i-1} : c(w_{i-1}, w) > 0\}\right|$$

- Normalized by the total number of bigram types

$$P_{CONTINUATION}(w) = \frac{\left|\{w_{i-1} : c(w_{i-1}, w) > 0\}\right|}{\left|\{(w_{j-1}, w_j) : c(w_{j-1}, w_j) > 0\}\right|}$$

Kneser-Ney Smoothing

$$P_{KN}(w_i \mid w_{i-1}) = \frac{\max(c(w_{i-1}, w_i) - d, 0)}{c(w_{i-1})} + \lambda(w_{i-1}) P_{CONTINUATION}(w_i)$$

λ is a **normalising constant** that is used to distribute the probability mass we've discounted:

$$\lambda(w_{i-1}) = \frac{d}{c(w_{i-1})} \left| \{ w : c(w_{i-1}, w) > 0 \} \right|$$

the normalised discount

The number of word types that can follow w_{i-1}
= # of word types we discounted
= # of times we applied normalised discount

Resources

- KenLM Language Model Toolkit:
  https://kheafield.com/code/kenlm/

- CMU Statistical Language Modeling Toolkit:
  http://www.speech.cs.cmu.edu/SLM/toolkit.html

- Google Books N-gram Counts:
  http://storage.googleapis.com/books/ngrams/books/datasetsv2.html

Reading

- Jurafsky, Daniel, and James H. Martin. 2024. Speech and Language Processing. 3rd edition. **Chapters 3.5-3.7.**